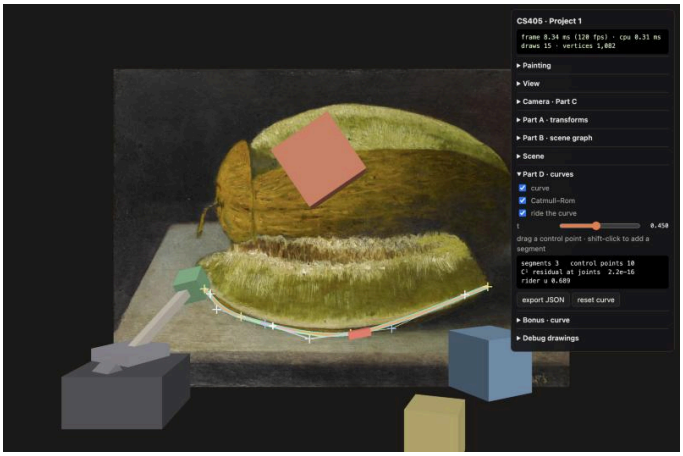
CS405 Project 1

Şefik Efe Altınoluk

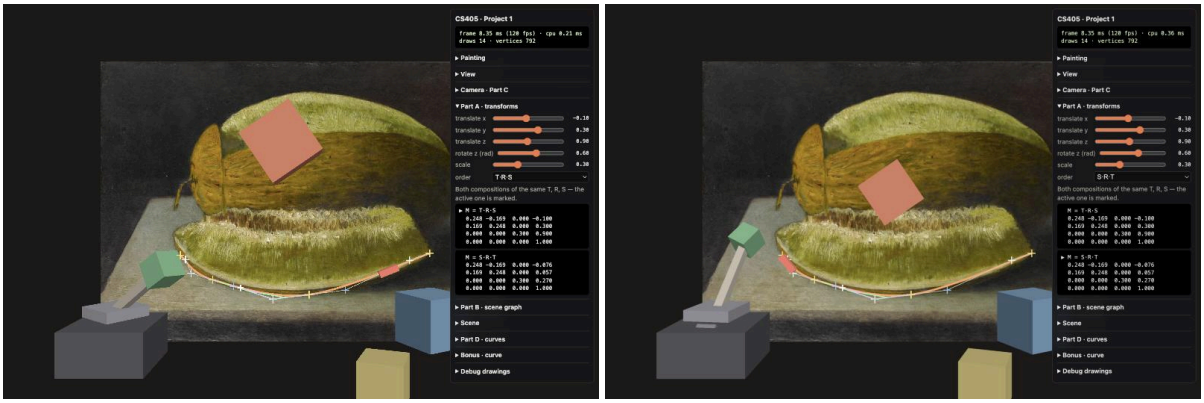
Painting: Şeker Ahmed Paşa, *Kavunlu Natürmort* (Sakıp Sabancı Museum).
I picked it for two reasons. The cut melon's lower edge is one long smooth line with no ambiguity about where it runs, which is what Part D needs, and the painting also contains a piece of hard geometry, the stone slab, whose three straight edges can be measured. That second property made the camera-matching bonus possible, so one painting covers both the curve work and the perspective work.



The application: the painting as a textured quad, Part A and B objects in front of it, and the traced melon contour with its control polygon and de Casteljau construction.

1. Part A, transformations

A matrix is a `Float32Array(16)` in column-major order: element (row r, column c) lives at index `c*4 + r`, which is exactly the layout WGSL wants for a `mat4x4f`, so the array goes into a uniform buffer untouched. Points are column vectors, $p' = Mp$, and `multiply(A, B)` is AB, so B acts first. The translation sits in the last column, at indices 12, 13, 14.
With $t = (-0.10, 0.30, 0.90)$, a rotation of 0.6 rad about z and a uniform scale of 0.3, both orders give the same linear 3×3 part, since a uniform S commutes with R. The translation columns differ: $(-0.100, 0.300, 0.900)$ for TRS against $(-0.076, 0.057, 0.270)$ for SRT. In TRS the rightmost factor acts first, so the object is scaled and rotated about its own origin and only then moved, which leaves the translation column equal to t. In SRT the object moves away from the origin first, so the later rotation swings it along an arc and the later scale shrinks the displacement along with the object. The position becomes SRt, and 0.900 becomes $0.900 \times 0.3 = 0.270$. Matrix multiplication does not commute, and the translation column is where you see it. The rest of the project uses TRS.



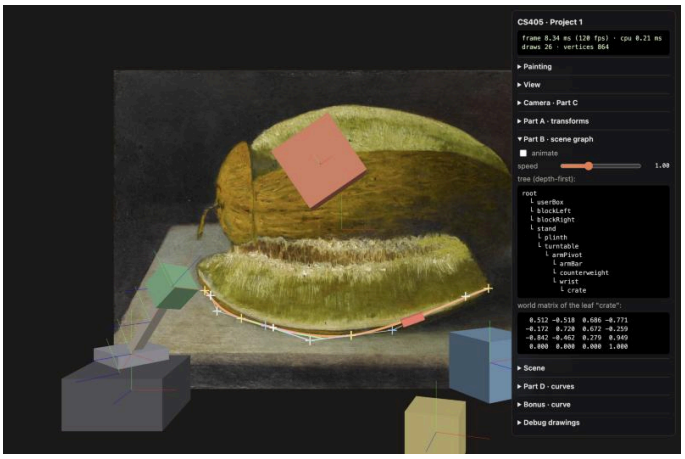
Left, $M = TRS$; right, $M = SRT$, built from the same T, R and S. The panel prints both matrices every frame and marks the active one with a triangle.
Rigid inverse. For $M = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$ with R orthonormal, $R^{-1} = R^T$, so $M^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0 & 1 \end{bmatrix}$. `invertRigid` writes that out directly: a transpose of the upper 3×3 and three dot products, with no determinant and no division. Part C uses it twice.

2. Part B, scene graph

Each node holds a local transform and a world matrix computed by one depth-first traversal, `world = parent.world · local`. Only three local matrices are animated and the rest follows from the traversal. I kept the node locals rigid (T·R only) and put a box's size in `node.data.size`, applied at draw time as `model = world · S`. Two things fall out of that: the world matrices stay rigid, so `invertRigid` works on them and they are readable enough to check by hand, and a parent's scale cannot squash the debug axes.

```
root
 └ userBox / blockLeft / blockRight   (Part A)
 └ stand                              level 1
    └ plinth                          level 2
    └ turntable  ← R_y(0.8t)          level 2
       └ armPivot ← R_z(.35+.45sin1.3t)  3
          └ armBar, counterweight        4
          └ wrist                        4
             └ crate ← R_y(−2t)          5
```

Five levels, twelve nodes.
The leaf world matrix, by hand. At $t = 0$ the animated rotations are the identity except for the arm's 0.35 rad bias, so the chain for `crate` collapses to



Part B: every node's local axes, the printed tree, and the leaf's live world matrix.

$$W = T(-1.05, -0.92, 0.70) \cdot T(0, 0.30, 0) \cdot R_z(0.35) \cdot T(0.52, 0, 0)$$

With $\cos 0.35 = 0.9394$ and $\sin 0.35 = 0.3429$, $R_z(0.35) \cdot (0.52, 0, 0) = (0.4885, 0.1783, 0)$, and the two leading translations add to $(-1.05, -0.62, 0.70)$. The crate's origin is therefore $(-0.5615, -0.4417, 0.700)$ and the rotation part is $R_z(0.35)$. The panel prints `0.9394 -0.3429 0 -0.5615 / 0.3429 0.9394 0 -0.4417 / 0 0 1 0.7000`, which agrees to every digit shown.

3. Part C, camera and projection

The camera looks down its own $-z$, so its $+z$ points backwards, from target to eye. Two cross products build an orthonormal basis:

$$\mathbf{f} = \frac{\mathbf{e} - \mathbf{t}}{\|\mathbf{e} - \mathbf{t}\|}, \qquad \mathbf{s} = \frac{\mathbf{up} \times \mathbf{f}}{\|\mathbf{up} \times \mathbf{f}\|}, \qquad \mathbf{u} = \mathbf{f} \times \mathbf{s}$$

$\mathbf{u}$ is orthogonal to both by construction, so no third normalisation is needed. The camera's placement is the rigid matrix $C = [\,\mathbf{s}\ \mathbf{u}\ \mathbf{f}\ |\ \mathbf{e}\,]$, the view matrix is its inverse, and the rigid-inverse identity of Part A gives

$$V = C^{-1} = \begin{bmatrix} s_x & s_y & s_z & -\mathbf{s}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ f_x & f_y & f_z & -\mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

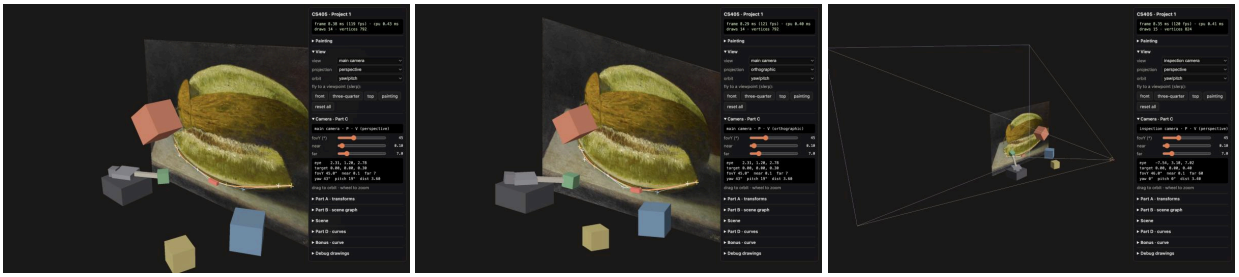
Two checks follow at once. $V\mathbf{e} = 0$, and $V\mathbf{t}$ has $z = \mathbf{f} \cdot (\mathbf{t} - \mathbf{e}) = -\|\mathbf{t} - \mathbf{e}\|$, so the target lands on the negative z axis at its true distance. Both are unit tests.

The degenerate case. If $\mathbf{up}$ is parallel to $\mathbf{f}$ the first cross product vanishes. The code detects $\|\mathbf{up} \times \mathbf{f}\| < 10^{-6}$ and substitutes whichever reference up is least aligned with $\mathbf{f}$. The orbit's pitch is clamped at exactly $\pm 90°$ so this path can be reached from the UI, and looking straight down still gives a finite, orthonormal view matrix.

Perspective for WebGPU. Clip space here is $z \in [0, 1]$, not OpenGL's $[-1, 1]$. Row 4 is $(0, 0, -1, 0)$, so $w_{\text{clip}} = -z_{\text{view}}$, which is the perspective divide itself. Writing row 3 as $(0, 0, A, B)$ and demanding that near maps to 0 and far to 1:

$$\frac{A(-n) + B}{n} = 0, \qquad \frac{A(-f) + B}{f} = 1 \qquad \Rightarrow \qquad A = \frac{f}{n - f}, \quad B = \frac{nf}{n - f}$$

The x and y rows are $\cot(\text{fovY}/2)/\text{aspect}$ and $\cot(\text{fovY}/2)$. `orthographic` uses the same near-to-0, far-to-1 convention but leaves $w = 1$. OpenGL's -1 to 1 formulas put half the scene behind the near plane, which is the first thing to check if objects vanish.


Left to right: perspective, orthographic from the same viewpoint, and the main camera's frustum seen from the inspection camera. The orthographic height is kept at $2d\tan(\text{fovY}/2)$, so both projections frame the same thing at the target and the two screenshots are directly comparable. The cubes give it away.

My first instinct on near and far was to keep the ratio small. That turned out to be wrong. With the matrix above, $z_{\text{ndc}}(d) = f(n - d)/(d(n - f))$, so

$$\left|\frac{dz_{\text{ndc}}}{dd}\right| = \frac{fn}{d^2(f - n)} \qquad \Rightarrow \qquad \Delta d = qd^2\frac{f - n}{fn} \to \frac{qd^2}{n} \quad (f \gg n)$$

for one quantum $q = 2^{-24}$ of a 24-bit depth buffer. The near plane sets the precision and far barely matters.

near, far	Δd at $d = 1$	Δd at $d = 5$
0.1, 7 (chosen)	5.9×10^{-7}	1.5×10^{-5}
0.1, 1000	6.0×10^{-7}	1.5×10^{-5}
0.01, 7	5.9×10^{-6}	1.5×10^{-4}

Widening far by a factor of 140 costs about 1 %, while dropping near by a factor of 10 costs exactly a factor of 10. So near is kept as large as the scene allows, 0.1 with the orbit clamped to at least 0.4, and far = 7 is chosen not for precision but so that nothing is ever clipped.

The inspection camera. A second, fixed camera watches the whole set-up. The eight frustum corners are trivial in view space, where the half-height at distance d is $d\tan(\mathrm{fovY}/2)$, or a constant in orthographic, so they are built there and pushed to world space with `invertRigid(V)`, reusing Part A. Dragging still orbits the main camera while the inspector watches, so the frustum can be seen moving.

4. Part D, curves over the painting

The basis is $B_0 = (1-t)^3$, $B_1 = 3(1-t)^2 t$, $B_2 = 3(1-t)t^2$, $B_3 = t^3$, the terms of $((1-t)+t)^3$, which is why they sum to 1 for every t. The curve is an affine combination of its control points and stays inside their convex hull.

Proof that $B(0) = P_0$ and $B(1) = P_3$. At $t = 0$ every factor of t vanishes, so $B_1 = B_2 = B_3 = 0$ and $B_0 = 1$, and the sum collapses to $1 \cdot P_0$. At $t = 1$ every factor of $(1-t)$ vanishes, so $B_0 = B_1 = B_2 = 0$ and $B_3 = 1$, giving P_3. ∎ The derivative $B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2)$ gives $B'(0) = 3(P_1 - P_0)$ and $B'(1) = 3(P_3 - P_2)$, so the end tangents lie along the first and last legs of the control polygon.

The C^1 condition at a joint. For segments $P_0P_1P_2P_3$ and $Q_0Q_1Q_2Q_3$ sharing a joint, C^0 needs $Q_0 = P_3$, and C^1 additionally needs the tangents to agree:

$$3(P_3 - P_2) = 3(Q_1 - Q_0) = 3(Q_1 - P_3) \qquad \Rightarrow \qquad Q_1 = 2P_3 - P_2$$

The outgoing handle is the incoming one mirrored through the joint. That is `curves.c1NextControl`, applied to every interior joint after any edit, so dragging cannot break C^1. The panel prints the worst tangent mismatch across all joints live, and it stays at about 2×10^{-16}, which is rounding.

`deCasteljau` repeatedly interpolates neighbouring points of the previous level until one is left, and returns every level so the construction can be drawn; the `t` slider is global, and the construction appears in whichever segment t falls in. Both second families are implemented. `catmullRom` passes through its control points with tangents $(P_{i+1} - P_{i-1})/2$, while `bsplineBasis` gives the uniform cubic B-spline weights, which at $t = 0$ are $(1, 4, 1)/6$: non-negative and summing to 1, but without interpolating, which is precisely how it differs. The app draws the Catmull–Rom spline through the Bézier's joints, wider and underneath, so it reads as a halo where the two agree.

The default curve traces the bottom contour of the cut melon. I found that contour by scanning the image column by column for the lowest pixel passing a melon-flesh test (blue below 55 % of the red/green mean), placing four joints on the result, and taking the handles from the Catmull–Rom-to-Bézier conversion, which is C^1 by construction. It follows the painted edge to about 1 % of the image width. Control points are dragged by intersecting a world-space ray with $z = 0$, using the same code for both projections, and a joint carries its handles with it. A box rides the curve oriented by the tangent, and since the curve is planar that is one $R_z(\arctan(t_y, t_x))$.

5. Part E, integration and performance

One application, one panel grouped by part. Every debug drawing has its toggle in the same place, `reset all` restores the camera, sliders, animation and curve, and the header shows smoothed frame time, fps, CPU time, draw calls and vertices. The code is split into `math/`, `scene/`, `render/` and `ui/`. Two things that catch people out are commented in place: every drawable owns its uniform buffer, because `queue.writeBuffer` calls all run before the frame's commands, and the line pipelines share one explicit bind-group layout, because with `layout: "auto"` a bind group made for one pipeline cannot be used with another.

Machine: Apple M4 Pro (20-core GPU, Metal 3), macOS 26.5.2, Chromium 152, canvas 1240 × 820 CSS px at devicePixelRatio 2 (2480 × 1640 device px). Figures are means over about 360 consecutive frames, and CPU time is the application's own counter.

Configuration	draws	vertices	frame	fps	CPU
Parts A to D, no debug drawing	14	792	8.33 ms	120	0.35 ms
plus node axes, Catmull–Rom, perspective overlay	28	1 188	8.33 ms	120	0.43 ms
plus surface of revolution	15	16 128	8.33 ms	120	0.52 ms
everything on at once	30	16 814	8.35 ms	120	0.61 ms

Every configuration is display-locked at 120 Hz, so the 8.33 ms is the refresh interval rather than the work. The scene is far too small to stress an M4 Pro, so the only figure that discriminates is CPU time, which stays between 0.2 and 0.6 ms. The frame is about 4 % CPU-bound at worst, and the 95th-percentile frame never exceeded 9.3 ms.

6. Bonus work

6.1 Least-squares fitting of a cubic Bézier

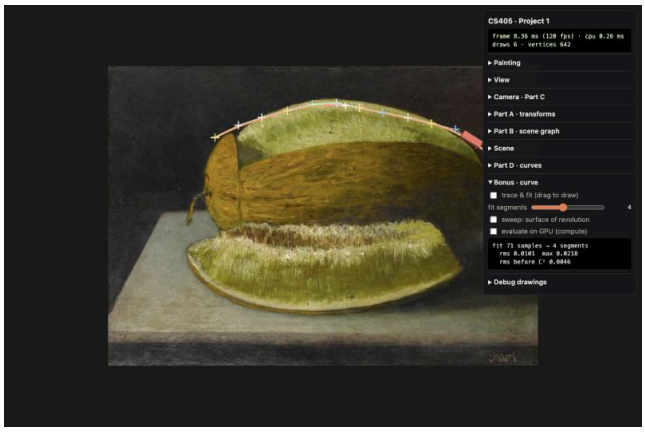
Control points enter the Bézier linearly, so with the endpoints pinned to the ends of a traced stroke the only unknowns are P_1 and P_2. Minimising $E = \sum_i \|B(t_i) - Q_i\|^2$ gives a 2×2 normal-equation system, the same one for every coordinate because the basis values are scalars. Parameters start from chord length; after each solve one Newton step on $(B(t) - Q) \cdot B'(t) = 0$ gives every sample the parameter of its closest point, and the system is solved again. Eight rounds bring a clean stroke to a few tenths of a percent of its own size. `fitPiecewise` cuts the stroke into pieces of equal arc length, fits each, then applies the C^1 mirror rule, which moves the curve, so the panel reports the error before and after. For the stroke in the figure (71 samples, 4 segments): rms 0.0046 before mirroring, 0.0101 after, maximum deviation 0.0218. In world units on a painting two units high that is 0.5 % and 1.1 %, so enforcing C^1 doubles the residual.

6.2 Surface of revolution swept from the curve

The curve is already a profile. Measured in the frame of the chord joining its ends ($\hat{\mathbf{a}}$ along the chord, $\hat{\mathbf{n}}$ perpendicular to it in the painting, $\hat{\mathbf{b}}$ out of it) and spun about $\hat{\mathbf{a}}$:

$$P(s, \theta) = O + s\hat{\mathbf{a}} + r(s)\big(\cos\theta\,\hat{\mathbf{n}} + \sin\theta\,\hat{\mathbf{b}}\big)$$

The tangents are $\partial_s P = \hat{\mathbf{a}} + r'(\cos\theta\,\hat{\mathbf{n}} + \sin\theta\,\hat{\mathbf{b}})$ and $\partial_\theta P = r(-\sin\theta\,\hat{\mathbf{n}} + \cos\theta\,\hat{\mathbf{b}})$, and their cross product is $r(r', -\cos\theta, -\sin\theta)$ in the local frame, so the outward unit normal is $(-r', \cos\theta, \sin\theta)/\sqrt{1 + r'^2}$, exact and with no finite differences. The chord runs along



A freehand stroke along the melon's upper contour, replaced by a four-segment C^1 fit.

the melon's own axis, so the sweep produces a melon: 36 sides by 71 profile samples, or 15 336 triangles, rebuilt whenever the curve changes.

6.3 Matching the camera to the painting's perspective

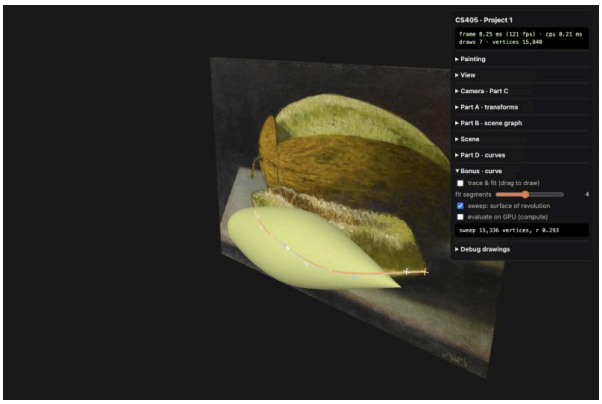
Three straight edges of the stone slab were located automatically, by taking the first sustained dark-to-light transition in each image column, and fitted by least squares in pixels on the 1280 × 893 image. Residuals of 1.5 to 2.3 px mean the edges really are straight.

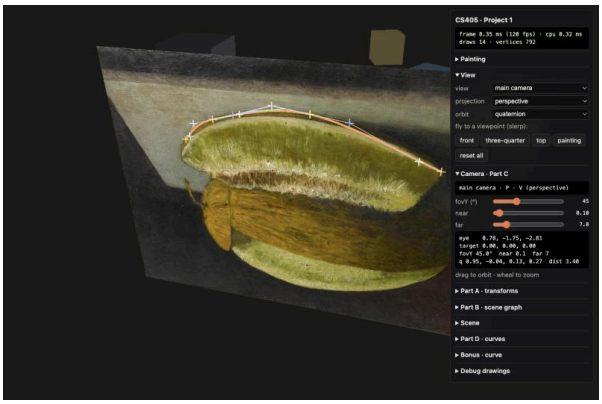
edge	slope	intercept	rms	columns
far (back of slab)	$+0.00902$	432.4	1.51 px	16
near (front of slab)	-0.00038	822.7	2.29 px	83
left (side of slab)	-1.90043	741.9	2.24 px	12

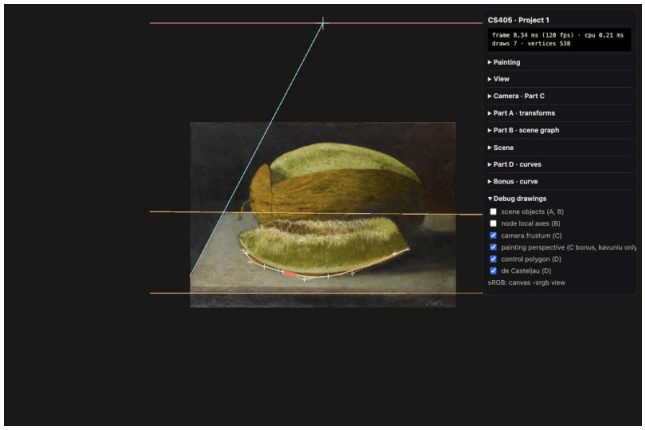
The near and far edges are parallel in the world, and in the image they meet 32 image widths to the right, so the slab's width axis is parallel to the picture plane. This is a one-point perspective and the painter's station point lies on the normal through the principal point. Assuming a centred, unrolled canvas, the depth vanishing point lies on the principal column, so intersecting the slab's left edge with $x = W/2$ places it at (640, –474) px, 921 px above the image centre and off the canvas, which is what a steeply downward view of a table looks like.

That offset satisfies $\Delta = f \tan\theta$, one equation in two unknowns. Recovering f needs two finite orthogonal vanishing points and the second here is at infinity, so the field of view is not recoverable from this painting. The project default fovY = 45° ($f = 1078$ px) gives an elevation of 40.5°, at about 0.75° per degree of fovY (33.0° at 35°, 47.0° at 55°). A perspective picture looks undistorted only from the normal through its principal point, at the distance where it fills the field of view it was drawn with; the quad is 2 units high, so that is $d = 1/\tan(\mathrm{fovY}/2) = 2.414$, where the `painting` viewpoint button flies to.

One consistency check fails. If the model were exact the width vanishing point would lie on the horizon at $y = -474$, which needs the back edge to rise 22 px across the image; it was measured falling 9 px, a 31 px disagreement well outside the 1.5 px residual. The painting is not a strict perspective projection, so I trusted the better conditioned side edge over the pair of nearly parallel ones.







The reconstruction over the painting: fitted slab edges (orange), the side edge extended (cyan), the horizon (magenta) and the vanishing point (white cross).

Left, §6.2: the traced contour swept into a solid of revolution. Right, §6.4: past the pole, where the free orbit has carried the camera behind and under the painting and the readout shows q instead of yaw and pitch.

6.4 Quaternion orbit with interpolation between viewpoints

`src/math/quat.js` implements axis-angle construction, the quaternion product, rotation of a vector by $v' = v + 2w(\mathbf{u} \times v) + 2\mathbf{u} \times (\mathbf{u} \times v)$ (two cross products, no matrix), and slerp along the shortest arc with a normalised-lerp fallback when $\sin\Omega \to 0$. In quaternion mode the camera's orientation is one unit quaternion: the eye is the rotated $+z$ at `distance` and `up` is the rotated $+y$, so no up vector is left to go singular. A drag left-multiplies a yaw about the world up and right-multiplies a pitch about the camera's own x. With no stored angles there is nothing to lock, so the orbit rolls over the pole and keeps going, where the yaw/pitch orbit of Part C stops at $\pm 90°$; switching models does not move the camera. The four viewpoint buttons fly there by slerp at a constant angular rate, with distance, target and fovY interpolated linearly under a smoothstep.

6.5 Evaluating the curve in a compute shader

`src/render/shaders/curve.wgsl` runs one invocation per sample. Each finds its segment, evaluates the cubic and its derivative in Bernstein form, and writes the two ribbon vertices there as six raw `f32`s rather than a struct, because a struct of two `vec3f` would be padded to 32 bytes and no longer match the vertex layout. The output buffer is created `STORAGE | VERTEX`, so the bytes the compute pass writes are read as vertices with no copy, and the compute work is submitted as its own command buffer before the frame's, which orders the write ahead of the read.

At this scale it does not pay off: tessellating the 145-sample curve costs 0.33 to 0.44 ms of frame CPU time on the CPU and 0.35 to 0.42 ms on the GPU, because submitting an extra command buffer every frame costs about as much as the JavaScript it removes. Both curves coincide, so the technique is correct; it would start to win in the thousands of samples.

7. What went wrong, and how I found it

Hairline curves. The first version drew everything with GPU `line-list` primitives, one *device* pixel wide, which is half a CSS pixel at devicePixelRatio 2. I fixed it by building ribbons: thin quad strips around each polyline, stitched into one triangle strip with duplicated, zero-area vertices.

Bind groups and `layout: "auto"`. A second line pipeline for `line-strip` produced `setPipeline: parameter 1 is not of type 'GPURenderPipeline'`, then validation failures. With `layout: "auto"` each pipeline gets its own bind-group layout, and a bind group built from one cannot be used with another even when the two are structurally identical. Fixed with one explicit `GPUBindGroupLayout`, shared.

The surface of revolution came out inside-out. It rendered as a flat silhouette with odd, uniform shading. $\partial_s P \times \partial_\theta P$ is the *inward* normal, so walking the (s, θ) grid in the obvious order winds every triangle backwards and `cullMode: "back"` keeps only the inside. I found it by reading `scene/geometry.js`, which sets the project's convention that $(v_1 - v_0) \times (v_2 - v_1)$ is outward, fixed it by swapping two vertices, and locked it down with a unit test on the winding.

The fit-error metric measured the wrong thing. A straight stroke fitted with a straight curve reported an error of 0.044. The fit was fine: `error()` compared each sample to the nearest *vertex* of a dense walk of the curve, so it could never read below the step size. Changed to the distance to the nearest *segment* of that polyline, after which the straight case reads 0.000.

I was wrong about near and far. A code comment claimed [0.1, 7] "keeps far more precision than a careless [0.1, 1000]". Working out $\Delta d = q d^2 (f - n)/(fn)$ showed that is false for a $[0, 1]$ depth mapping: far changes the answer by about 1 %, near by the full factor. I corrected the comment and the README, and §3 now gives the real reason $f = 7$ was chosen.

"1 fps". Early performance readings were absurd, 350 to 1000 ms per frame. The application was fine: `requestAnimationFrame` is throttled to about 1 Hz when the window is in the background, which is where the automated browser pane was. Re-measured in the foreground, a steady 120 fps.

Appendix A. Exported curve JSON

Control points in normalised painting coordinates (u right, v down, origin top-left), as written by `export JSON` and checked in as `curve-kavunlu.json`. `points` holds the joints, which double as the Catmull–Rom control points; each interior joint also appears twice in `segments`, ending one and starting the next, which is the C^0 half of the joint condition of §4.


```
{ "painting": "kavunlu", "kind": "bezier-c1",
  "segments": [ [[0.2,0.691],[0.213,0.706],[0.227,0.755],[0.28,0.778]],
                [[0.28,0.778],[0.333,0.801],[0.43,0.847],[0.52,0.831]],
                [[0.52,0.831],[0.61,0.815],[0.77,0.709],[0.82,0.684]] ],
  "points": [[0.2,0.691],[0.28,0.778],[0.52,0.831],[0.82,0.684]] }
```


Appendix B. AI use

I used Claude (Anthropic) to help me draft the general outline of ui codes, understanding the JS syntax where I lacked language wise information, and to add the report scientific markdown style and proper visual structure. I used AI in the UI wiring which helped me to focus on the core of the project rather dealing with ui bugs and implementations. It also did the measurement work, meaning the scans of the painting for the melon contour and the slab edges, the line fits and the frame-time benchmarks. Every item in §7 was found during that process, and the winding bug, the bind-group-layout bug and the error-metric bug were diagnosed with its help, each followed by a new unit test.

Additionally, I used AI to write new unit test files to check the coverage of the bonus parts that I implemented: see `quat.test.js`, `revolve.test.js` and `fit.test.js`. Finally, I used AI to assist me in writing and documenting a proper README.md with setup and project explanation details.